

Week 42: From Backpropagation to Differential Equations

Morten Hjorth-Jensen
Viktor Svensson

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

Plan for today

First hour

1. **Backpropagation**: the chain rule, organised
2. **Activation and cost functions**: why they come in pairs
3. **Universal approximation**: what a network can represent

Second hour

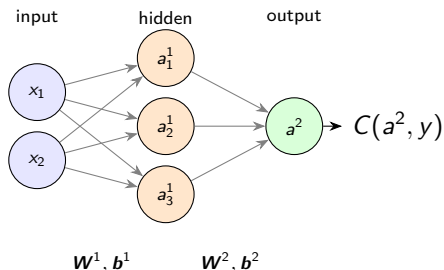
4. **Neural networks for differential equations**
 - ▶ the equation as the cost
 - ▶ why the activation function matters
 - ▶ boundary conditions
 - ▶ pros and cons vs. standard methods

On the board (take notes)

Most derivations today are done on the blackboard. The slides give the setup and the results.

Tuesday: backpropagation in code. **Exercises week 42**: deadline Friday October 16.

Recap: the forward pass



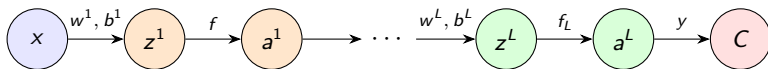
One sample $\mathbf{x}$, layers $l = 1, \dots, L$, $\mathbf{a}^0 = \mathbf{x}$:

$$\mathbf{z}^l = \mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l, \quad \mathbf{a}^l = f(\mathbf{z}^l)$$

- ▶ $\mathbf{W}^l$ has shape (nodes out, nodes in)
- ▶ f acts element by element; the last layer has its own *output* activation f_L
- ▶ the cost C compares the output $\mathbf{a}^L$ with the target $\mathbf{y}$

In code with many samples we stack them as rows, $\mathbf{Z}^l = \mathbf{A}^{l-1} \mathbf{W}^l + \mathbf{b}^l$, so $\mathbf{W}^l$ is stored as (in, out): the same equations, transposed.

Backpropagation with one node per layer



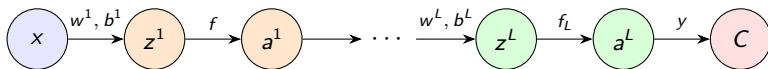
All scalars: $z^l = w^l a^{l-1} + b^l$,
 $a^l = f(z^l)$, $a^0 = x$. The *error* of layer
 l is

$$\delta^l = \frac{\partial C}{\partial z^l}.$$

On the board (W1)

Chain rule along the chain: δ^L , then
 δ^l from δ^{l+1} , then $\partial C / \partial w^l$ and
 $\partial C / \partial b^l$.

Backpropagation with one node per layer



All scalars: $z^l = w^l a^{l-1} + b^l$,
 $a^l = f(z^l)$, $a^0 = x$. The error of layer l is

$$\delta^l = \frac{\partial C}{\partial z^l}.$$

On the board (W1)

Chain rule along the chain: δ^L , then δ^l from δ^{l+1} , then $\partial C / \partial w^l$ and $\partial C / \partial b^l$.

Result: one chain-rule factor per line

$$\delta^L = f'_L(z^L) \frac{\partial C}{\partial a^L} \quad \partial C / \partial a^L \cdot da^L / dz^L$$

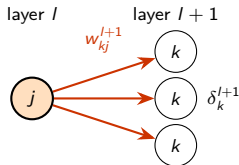
$$\delta^l = \delta^{l+1} w^{l+1} f'(z^l) \quad \partial C / \partial z^{l+1} \cdot \partial z^{l+1} / \partial a^l \cdot da^l / dz^l$$

$$\frac{\partial C}{\partial w^l} = \delta^l a^{l-1} \quad \partial C / \partial z^l \cdot \partial z^l / \partial w^l$$

$$\frac{\partial C}{\partial b^l} = \delta^l \quad \partial C / \partial z^l \cdot 1$$

- ▶ Start at the output, go *backwards*: each δ^l reuses δ^{l+1} .
- ▶ The error picks up a factor $w f'$ in every layer.

Many nodes per layer: where the matrices come in

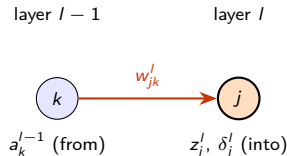


Node j feeds every node k of layer $l+1$, so its error comes back along every edge:

$$\delta_j^l = \sum_k w_{kj}^{l+1} \delta_k^{l+1} f'(z_j^l)$$

The sum runs over the *first* index of $\mathbf{W}^{l+1}$, so in matrix form it is $(\mathbf{W}^{l+1})^T \boldsymbol{\delta}^{l+1}$.

Shape check: $\mathbf{W}^{l+1}$ is (n_{l+1}, n_l) , so $(\mathbf{W}^{l+1})^T \boldsymbol{\delta}^{l+1}$ has n_l entries, and $\boldsymbol{\delta}^l (\mathbf{a}^{l-1})^T$ is (n_l, n_{l-1}) , the shape of $\mathbf{W}^l$.

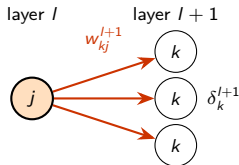


One weight, one product:

$$\frac{\partial \mathcal{C}}{\partial w_{jk}^l} = \delta_j^l a_k^{l-1} \quad \Rightarrow \quad \frac{\partial \mathcal{C}}{\partial \mathbf{W}^l} = \boldsymbol{\delta}^l (\mathbf{a}^{l-1})^T,$$

an outer product.

Many nodes per layer: where the matrices come in



Node j feeds every node k of layer $l+1$, so its error comes back along every edge:

$$\delta_j^l = \sum_k w_{kj}^{l+1} \delta_k^{l+1} f'(z_j^l)$$

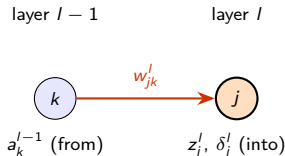
The sum runs over the *first* index of $\mathbf{W}^{l+1}$, so in matrix form it is $(\mathbf{W}^{l+1})^T \boldsymbol{\delta}^{l+1}$.

Shape check: $\mathbf{W}^{l+1}$ is (n_{l+1}, n_l) , so $(\mathbf{W}^{l+1})^T \boldsymbol{\delta}^{l+1}$ has n_l entries, and $\boldsymbol{\delta}^l (\mathbf{a}^{l-1})^T$ is (n_l, n_{l-1}) , the shape of $\mathbf{W}^l$.

The four equations ($\circ$: element-wise product)

$$\delta^L = f'_L(\mathbf{z}^L) \circ \frac{\partial C}{\partial \mathbf{a}^L}, \quad \delta^l = ((\mathbf{W}^{l+1})^T \boldsymbol{\delta}^{l+1}) \circ f'(\mathbf{z}^l), \quad \frac{\partial C}{\partial \mathbf{W}^l} = \delta^l (\mathbf{a}^{l-1})^T, \quad \frac{\partial C}{\partial \mathbf{b}^l} = \delta^l$$

In code (samples in rows) the same equations appear transposed, e.g. $\partial C / \partial \mathbf{W}^l = (\mathbf{A}^{l-1})^T \boldsymbol{\Delta}^l$.

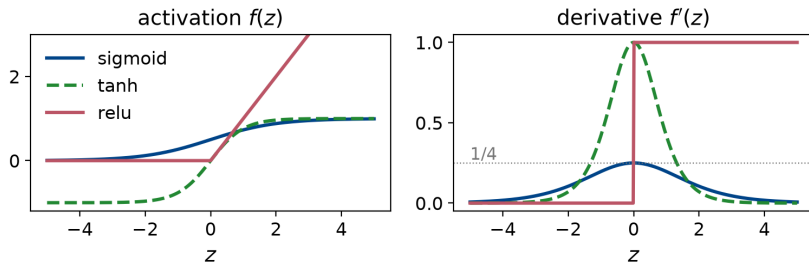


One weight, one product:

$$\frac{\partial C}{\partial w_{jk}^l} = \delta_j^l a_k^{l-1} \Rightarrow \frac{\partial C}{\partial \mathbf{W}^l} = \boldsymbol{\delta}^l (\mathbf{a}^{l-1})^T,$$

an outer product.

Hidden activation functions



Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$
 $\sigma' = \sigma(1 - \sigma) \leq \frac{1}{4}$.
Flat ($\sigma' \approx 0$) for large $|z|$.

tanh
 $\tanh' = 1 - \tanh^2 \leq 1$.
Centred at zero; also flat for large $|z|$.

ReLU $\max(0, z)$
 $f' = 1$ for $z > 0$, 0 otherwise. A unit with $z < 0$ for all inputs never learns (“dead”).

What matters

f must be **non-linear**: otherwise the whole network is a single linear map. For training, f' matters. Backpropagation multiplies by f' once per layer, so with L sigmoid layers the f' factors alone multiply the error by at most $(\frac{1}{4})^L$: the *vanishing gradient*. This is why ReLU is the default in deep networks.

Quick question 1: why not start at zero?

Question

A network starts with *all* weights and biases equal to zero. Look at the equations: which gradients are non-zero after the first backward pass? What happens after that?

$$\delta^l = ((\mathbf{W}^{l+1})^T \delta^{l+1}) \circ f'(z^l)$$
$$\frac{\partial C}{\partial \mathbf{b}^l} = \delta^l, \quad \frac{\partial C}{\partial \mathbf{W}^l} = \delta^l (\mathbf{a}^{l-1})^T$$

Quick question 1: why not start at zero?

Question

A network starts with *all* weights and biases equal to zero. Look at the equations: which gradients are non-zero after the first backward pass? What happens after that?

$$\delta^l = ((\mathbf{W}^{l+1})^T \delta^{l+1}) \circ f'(z^l)$$
$$\frac{\partial C}{\partial \mathbf{b}^l} = \delta^l, \quad \frac{\partial C}{\partial \mathbf{W}^l} = \delta^l (\mathbf{a}^{l-1})^T$$

Answer

Only the **output layer**.

- ▶ $\mathbf{W}^{l+1} = \mathbf{0}$ gives $\delta^l = \mathbf{0}$ in every hidden layer, whatever f is: no hidden weight or bias moves.
- ▶ Learning has to creep backwards from the output, one layer per step, and every gradient is a product of near-zero weights. Slow to start at best; if $f(0) = 0$ the network never leaves $\mathbf{W} = \mathbf{0}$.
- ▶ **Symmetry**: all units in a layer start identical, so they get identical gradients and stay copies of each other. The same happens for any equal non-zero start.

Random weights break the symmetry.

Initialising the weights

Random, to break the symmetry. **The scale matters.** $\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}$, with independent zero-mean entries of variance σ_w^2 in $\mathbf{W}$ and σ_b^2 in $\mathbf{b}$:

$$\mathbb{E}[\mathbf{z}\mathbf{z}^T] = \mathbb{E}[\mathbf{W}\mathbf{a}\mathbf{a}^T\mathbf{W}^T] + \mathbb{E}[\mathbf{b}\mathbf{b}^T] = (M\sigma_w^2\overline{\mathbf{a}^2} + \sigma_b^2)\mathbf{I}$$

Usually $\mathbf{b} = \mathbf{0}$ at the start, so $\text{Var}(\mathbf{z}) = M\sigma_w^2\overline{\mathbf{a}^2}$.

- ▶ σ_w too large: $|\mathbf{z}|$ large, tanh/sigmoid saturate, gradients blow up or die.
- ▶ σ_w too small: the signal shrinks in every layer, and so do the gradients.

Rule: keep $\text{Var}(\mathbf{z})$ the same in every layer

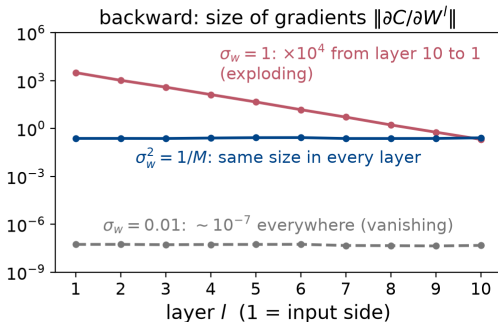
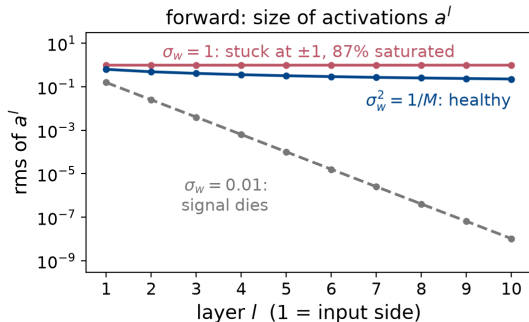
$$\sigma_w^2 = \frac{1}{M} \text{ (tanh)}, \quad \sigma_w^2 = \frac{2}{M} \text{ (ReLU)}$$

ReLU sets half of its inputs to zero, hence the factor 2 (“He initialisation”). Sigmoid: $\sigma' \leq \frac{1}{4}$, so with $1/M$ the gradients still shrink layer by layer.

The right scale depends on the *activation function*, and it decides whether *gradients* survive the backward pass.

The wrong scale breaks a deep network before training starts

A deep network: **10 hidden layers of 256 tanh units**, one linear output unit, no biases. Input: 500 random Gaussian vectors of length 256; targets: random numbers; cost $C = \frac{1}{2} (\overline{a^L} - y)^2$. Nothing is trained: one forward and one backward pass.



- ▶ **Too large:** saturated units and exploding gradients. **Too small:** the signal dies and the gradients vanish. Either way, gradient descent cannot train the network.
- ▶ $\sigma_w^2 = 1/M$: signal and gradients keep their size through all 10 layers.

The right scale depends on the activation function

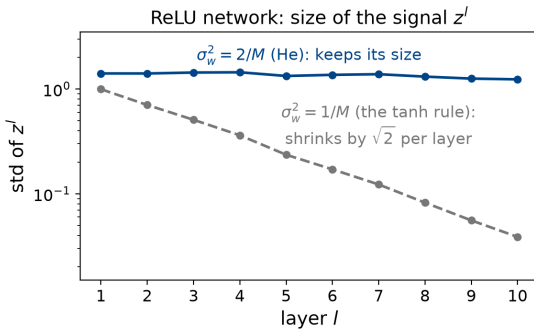
The same network with **ReLU** instead of tanh, initialised with the rule that worked for tanh, $\sigma_w^2 = 1/M$:

- ▶ The signal shrinks by $\sqrt{2}$ in *every* layer: 1/23 of its size after 10 layers, $\sim 10^{-7}$ after 50.
- ▶ No error message. The network just trains slowly, or not at all.

Why: ReLU sets half of its inputs to zero, so it passes on only half the variance. He initialisation, $\sigma_w^2 = 2/M$, makes up for it.

Take-away

An initialisation rule belongs to an activation function. Change the activation, and check the scale again.



Output activation and cost function come in pairs

Task	Output activation	Cost (one sample)	δ^L
regression, $y \in \mathbb{R}$	linear, $a = z$	$\frac{1}{2}(a - y)^2$	
binary, $y \in \{0, 1\}$	sigmoid, $a = \sigma(z)$	$-y \ln a - (1 - y) \ln(1 - a)$	
K classes, $\mathbf{y}$ one-hot	softmax, $a_k = \frac{e^{z_k}}{\sum_l e^{z_l}}$	$-\sum_k y_k \ln a_k$	

- ▶ The **output activation** gives the right *range*: any real number, a probability, or K probabilities that sum to one.
 - ▶ The **cost** matches it: each is the negative log-likelihood of a Gaussian, Bernoulli or categorical distribution.
- For these pairs, $\delta^L = f'_L(z^L) \partial C / \partial a^L$. What happens to f'_L ?

Output activation and cost function come in pairs

Task	Output activation	Cost (one sample)	δ^L
regression, $y \in \mathbb{R}$	linear, $a = z$	$\frac{1}{2}(a - y)^2$	$a - y$
binary, $y \in \{0, 1\}$	sigmoid, $a = \sigma(z)$	$-y \ln a - (1 - y) \ln(1 - a)$	$a - y$
K classes, $\mathbf{y}$ one-hot	softmax, $a_k = \frac{e^{z_k}}{\sum_l e^{z_l}}$	$-\sum_k y_k \ln a_k$	$\mathbf{a} - \mathbf{y}$

- ▶ The **output activation** gives the right *range*: any real number, a probability, or K probabilities that sum to one.
- ▶ The **cost** matches it: each is the negative log-likelihood of a Gaussian, Bernoulli or categorical distribution.

Softmax is not element-wise: there, f' is the Jacobian $\partial a_k / \partial z_j$, but the same cancellation happens.

For these pairs, $\delta^L = f'_L(z^L) \partial C / \partial a^L$. What happens to f'_L ?

f'_L cancels

For all three pairs, f'_L *cancels* against $\partial C / \partial a$. Sigmoid: $\sigma' = a(1 - a)$ and $\frac{\partial C}{\partial a} = \frac{a - y}{a(1 - a)}$, so

$$\delta^L = \mathbf{a}^L - \mathbf{y}$$

prediction minus target.

What goes wrong with a mismatched pair: sigmoid + squared error

Sigmoid output $a = \sigma(z)$, target $y = 1$:

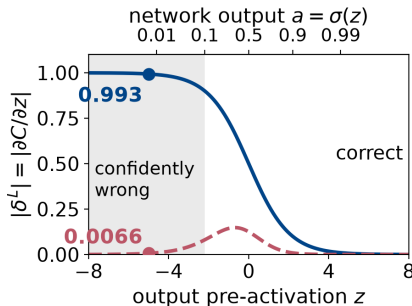
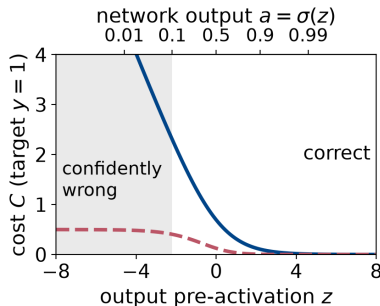
cross entropy: $\delta_{\text{CE}}^L = a - 1$

squared error: $\delta_{\text{SE}}^L = (a - 1) \underbrace{a(1 - a)}_{\sigma'(z)}$

Here σ' does *not* cancel, and $\sigma' \rightarrow 0$ as $a \rightarrow 0$.

At $z = -5$: δ_{SE}^L is **150× smaller** than δ_{CE}^L .

Squared error: the more confidently wrong, the *less* it learns.



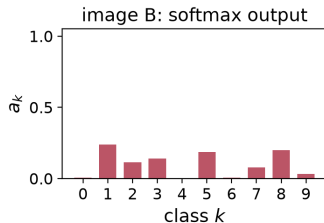
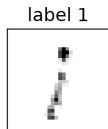
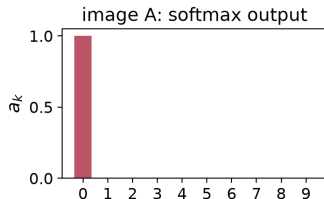
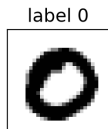
— cross entropy: $C = -\ln a$, $\delta^L = a - 1$

- - - squared error: $C = \frac{1}{2}(a - 1)^2$, $\delta^L = (a - 1)a(1 - a)$

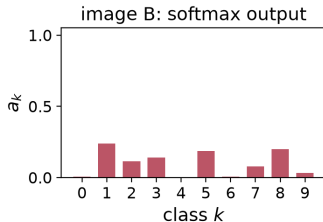
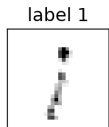
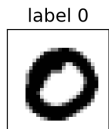
Quick question 2: which image trains the network?

Question

A network trained on handwritten digits (softmax + cross entropy) sees images A and B. The error $\delta^L = \mathbf{a}^L - \mathbf{y}$ has one component per class $0, \dots, 9$; $\mathbf{y}$ is the one-hot label. Which image changes the weights more?



Quick question 2: which image trains the network?



Question

A network trained on handwritten digits (softmax + cross entropy) sees images A and B. The error $\delta^L = \mathbf{a}^L - \mathbf{y}$ has one component per class $0, \dots, 9$; $\mathbf{y}$ is the one-hot label. Which image changes the weights more?

Answer

A, label 0: $\mathbf{a} \approx \mathbf{y} = (1, 0, \dots, 0)$, so $\delta^L \approx \mathbf{0}$: nothing to learn.

B, label 1: $\mathbf{y} = (0, 1, 0, \dots, 0)$.

- ▶ $\delta_1 = a_1 - 1 = 0.24 - 1 = -0.76$: the only negative component.
- ▶ Every other $\delta_k = a_k - 0 = a_k > 0$.

Raise the score of class 1, lower the rest. The network learns from what it gets wrong or is unsure about.

The universal approximation theorem

In words

A network with **one hidden layer** and a **non-linear** (non-polynomial) activation can approximate **any continuous function** on a bounded domain **as accurately as we want**, if the hidden layer is wide enough.

What it says

- ▶ A network *can* represent the function we are after.
- ▶ The non-linearity is essential.

What it does *not* say

- ▶ *How many* units are needed.
- ▶ Whether training *finds* the weights.
- ▶ How well it generalises to new data.

Why deep networks, then? For many functions, depth does the same job with far fewer units.

Why it works, in one dimension: ReLU units are kinks

One ReLU unit $\vee \text{ReLU}(wx + b)$ is zero, then a straight line: one *kink*, at $x = -b/w$.

On the board (W2)

Build a “hat” from three ReLU units. Then build any curve from hats.

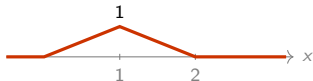
Why it works, in one dimension: ReLU units are kinks

One ReLU unit $\vee \text{ReLU}(wx + b)$ is zero, then a straight line: one *kink*, at $x = -b/w$.

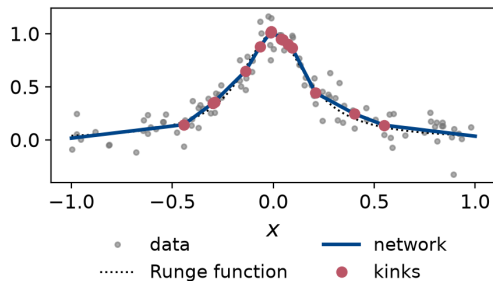
On the board (W2)

Build a “hat” from three ReLU units. Then build any curve from hats.

$$\text{ReLU}(x) - 2\text{ReLU}(x - 1) + \text{ReLU}(x - 2)$$



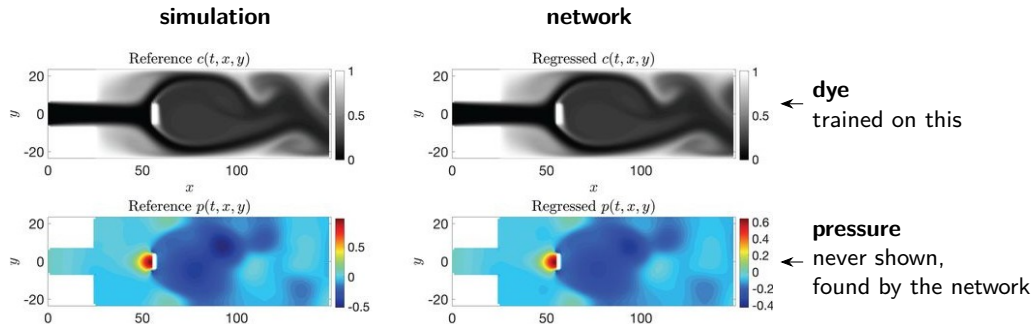
Hats at N points = connect-the-dots. More units, smaller error.



Training finds this by itself: a network with 50 ReLU units fitted to noisy data from the Runge function. The output is piecewise linear, and the kinks (red) sit where the function bends.

Motivation: seeing the invisible

We can film dye moving in a flow. Can a network work out the *pressure*?



Application: blood flow in brain aneurysms. Pressure and stress on the vessel wall from a medical scan, with no probe inside the patient.

M. Raissi, A. Yazdani, G. E. Karniadakis, *Hidden fluid mechanics*, Science **367**, 1026 (2020).

A network without data: the equation is the cost

Problem: $g'(x) = -\kappa g(x)$, $g(0) = g_0$. Nobody gives us data $g(x_i)$, only the equation.

Three ingredients:

1. A **trial solution** that satisfies the conditions for *any* network N :

$$g_t(x) = h(x) + B(x) N(x),$$

h satisfies the conditions, $B = 0$ where they are imposed.

2. The **residual**: left side minus right side of the equation, $r(x) = g'_t(x) - (-\kappa g_t(x))$, zero for the exact solution.
3. The **cost**: the mean squared residual at points x_i we choose ourselves (*collocation points*),

$$C = \frac{1}{n} \sum_{i=1}^n r(x_i)^2.$$

What is new

The residual contains g'_t , so we need $\partial N / \partial x$: a derivative with respect to the *input*, not the weights. It is just as easy to compute as the derivatives with respect to the weights.

What is not new

Network, optimiser and backpropagation for the weights. It is regression with “target” $r = 0$.

Example: exponential decay

$g' = -\kappa g$, $g(0) = g_0$, with $\kappa = 2$, $g_0 = 10$. Exact solution: $g = g_0 e^{-\kappa x}$.

On the board (W3)

Build g_t , compute g'_t , write the residual and the cost.

Example: exponential decay

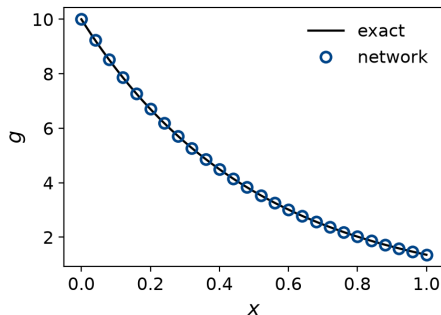
$g' = -\kappa g$, $g(0) = g_0$, with $\kappa = 2$, $g_0 = 10$. Exact solution: $g = g_0 e^{-\kappa x}$.

On the board (W3)

Build g_t , compute g'_t , write the residual and the cost.

$$g_t = g_0 + x N(x), \quad r = \underbrace{N + x N'}_{g'_t} - (-\kappa (g_0 + x N))$$

$g_t(0) = g_0$ whatever the network does.



Two hidden layers of 40 tanh units, 50 collocation points: max error 3×10^{-3} .

The activation function matters even more now

Poisson	diffusion	wave
$-g''(x) = f(x)$	$\partial_t g = \partial_x^2 g$	$\partial_t^2 g = c^2 \partial_x^2 g$

All three contain a *second* derivative, so we need $N''(x)$.

On the board (W4)

Differentiate a one-hidden-layer network

$N(x) = \sum_j v_j f(w_j x + b_j) + c$ twice with respect to x .

The activation function matters even more now

Poisson	diffusion	wave
$-g''(x) = f(x)$	$\partial_t g = \partial_x^2 g$	$\partial_t^2 g = c^2 \partial_x^2 g$

All three contain a *second* derivative, so we need $N''(x)$.

On the board (W4)

Differentiate a one-hidden-layer network

$N(x) = \sum_j v_j f(w_j x + b_j) + c$ twice with respect to x .

$$N''(x) = \sum_j v_j w_j^2 f''(w_j x + b_j)$$

ReLU: $f'' = 0$, so $N'' = 0$

A ReLU network is piecewise linear: *no curvature*, for any weights. It cannot solve $-g'' = f$.

$\max |N''|$ of a randomly initialised network:

tanh	2.9
GELU	1.2
sigmoid	0.06
ReLU	0

We use tanh: smooth, and $f' = 1 - f^2$, $f'' = -2ff'$ are cheap.

Boundary conditions built into the trial solution

Recipe: $g_t = h(x) + B(x) N(x)$, where h satisfies the conditions and $B = 0$ wherever a condition is imposed.

Conditions	Trial solution g_t
$g(0) = g(1) = 0$	$x(1-x) N$
$g(0) = a, g(1) = b$	$a + (b-a)x + x(1-x) N$
$g(0) = g_0, g'(0) = v_0$	$g_0 + v_0 x + x^2 N$

Check one row: put $x = 0$ and $x = 1$ (for the last row, also differentiate once). The N term drops out every time.

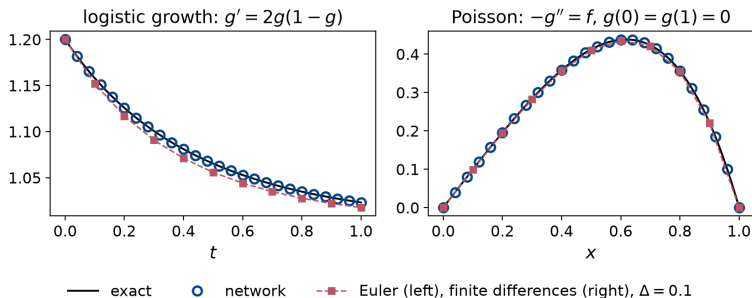
The good

The conditions hold *exactly*, for any weights: at the start, and after every step. The optimiser only has to solve the equation.

The bad

g_t must be designed by hand for every problem. A condition on the derivative, like $g'(1) = 0$, needs a trick. A curved boundary in 2D or 3D: impractical.

Results against standard methods

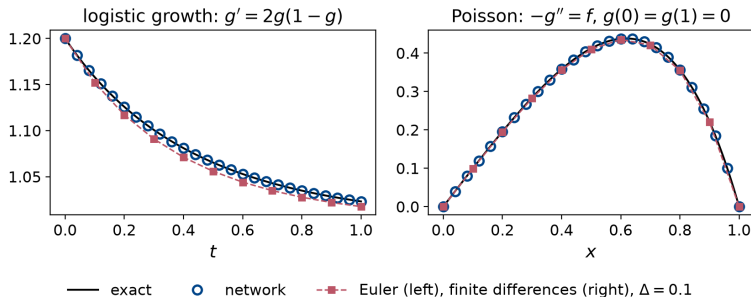


Logistic growth is a *non-linear* equation, but has the same trial solution $g_0 + xN$; only the residual changes. (Plots: $\Delta = 0.1$, so the points are visible.)

Guess first

Network or standard method: which is more *accurate*? Which is *faster*?

Results against standard methods



Logistic growth is a *non-linear* equation, but has the same trial solution $g_0 + xN$; only the residual changes. (Plots: $\Delta = 0.1$, so the points are visible.)

Logistic, max error		Poisson, max error	
Euler, $\Delta t = 0.01$	9×10^{-4}	FD, $\Delta x = 0.01$	2×10^{-5}
RK4, $\Delta t = 0.01$	2×10^{-10}		
network	1×10^{-4}	network	1×10^{-5}

Read it honestly

The network beats Euler, but a better method (RK4) is far more accurate. And the standard methods take *microseconds*, the network thousands of training steps. For simple equations like these the standard methods win.

A partial differential equation: diffusion

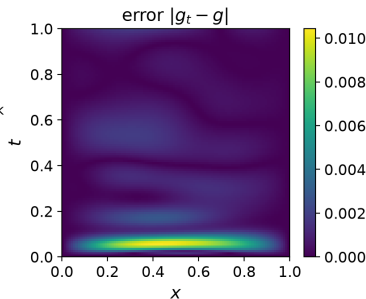
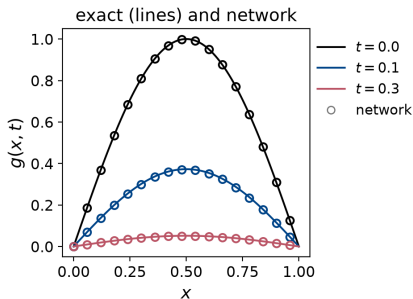
$$\frac{\partial g}{\partial t} = \frac{\partial^2 g}{\partial x^2}, \quad g(0, t) = g(1, t) = 0, \\ g(x, 0) = \sin \pi x.$$

Three conditions, one trial solution:

$$g_t = (1 - t) \sin \pi x + x(1 - x) t N(x, t)$$

The network now has *two inputs*, $N(x, t)$.
Nothing else changes.

Check: $t = 0$ gives $\sin \pi x$; $x = 0$ and $x = 1$ give 0.



Error exactly zero on the three edges (built into g_t), largest near $t = 0$ where the solution changes fastest: $\max 1 \times 10^{-2}$. Finite differences: 1.5×10^{-3} , in milliseconds.

Soft constraints: physics-informed neural networks (PINNs)

Instead of building g_t by hand: let the network *be* the solution (u , like g before), $u \approx N(x, t)$, and add the conditions to the cost as penalties:

$$L = L_{\text{PDE}} + \lambda(L_{\text{IC}} + L_{\text{BC}})$$

$$L_{\text{PDE}} = \frac{1}{n} \sum_i (\partial_t N - \partial_x^2 N)^2 \quad \text{inside the domain}$$

$$L_{\text{IC}} = \frac{1}{n} \sum_i (N(x_i, 0) - \sin \pi x_i)^2 \quad \text{at } t = 0$$

$$L_{\text{BC}} = \frac{1}{n} \sum_i (N(0, t_i)^2 + N(1, t_i)^2) \quad \text{at } x = 0, 1$$

A new condition is just a new term. λ weighs the terms, like the penalty in Ridge regression.

	RMSE	error at $t = 0$
hard (g_t)	1.8×10^{-3}	0 exactly
soft, $\lambda = 10$	6.5×10^{-3}	6×10^{-3}

Hard or soft?

- ▶ If you *can* build a trial solution, do: exact and more accurate.
- ▶ Soft is flexible, but the conditions hold only approximately.
- ▶ λ must be tuned. Too large, and the optimiser neglects the equation.

Quick question 3: a small residual, a wrong answer

Question

We solve $\partial_t u = \partial_x^2 u$ with the soft cost $L = L_{\text{PDE}} + \lambda(L_{\text{IC}} + L_{\text{BC}})$, but use a **ReLU** network $u = N(x, t)$. What does training converge to?

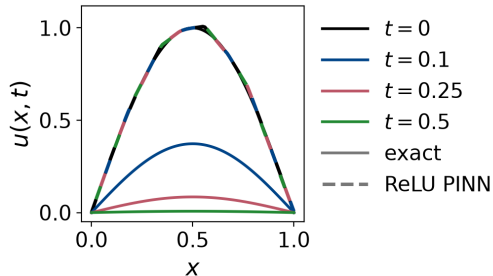
Quick question 3: a small residual, a wrong answer

Question

We solve $\partial_t u = \partial_x^2 u$ with the soft cost $L = L_{\text{PDE}} + \lambda(L_{\text{IC}} + L_{\text{BC}})$, but use a **ReLU** network $u = N(x, t)$. What does training converge to?

Answer

- ▶ ReLU gives $\partial_x^2 N = 0$ at every sample, so L_{PDE} only asks for $\partial_t N = 0$.
- ▶ The network learns $u \approx \sin \pi x$ for all t : it never diffuses. Small cost (6×10^{-4}), wrong answer: RMSE 0.65 (vs. 1.8×10^{-3}).
- ▶ **A small residual is not a solution.** Use smooth activations, and check against something you know.



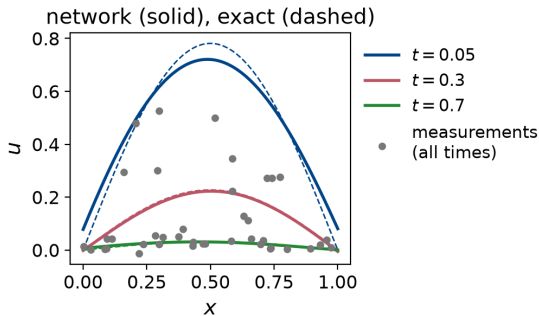
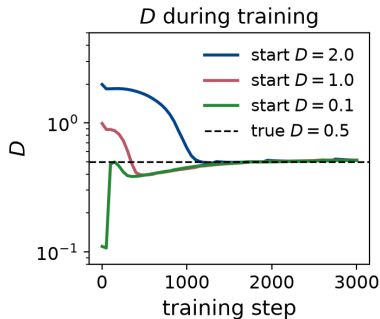
Why soft constraints at all? Inverse problems

$\partial_t u = D \partial_x^2 u$ with D **unknown**. No initial or boundary conditions, only 40 noisy measurements y_i . Treat D as one more trainable parameter:

$$L(N, D) = \frac{1}{n} \sum_j (\partial_t N - D \partial_x^2 N)_j^2 + \lambda \frac{1}{40} \sum_i (N(x_i, t_i) - y_i)^2$$

No trial solution is possible here: soft constraints are the only option.

True $D = 0.5$. Three different starting guesses all give $D \approx 0.51$. **But** with five times more noise: $D = 0.69$. A recovered parameter needs an error bar.



Neural networks versus standard methods

Advantages

- ▶ **No mesh:** complicated geometries are easy.
- ▶ **High dimensions:** a mesh grows exponentially with the dimension, random points do not.
- ▶ **The solution is a function:** evaluate and differentiate it anywhere.
- ▶ **Non-linear equations** need no extra work.
- ▶ **Data and equations together:** unknown parameters, inverse problems.

Disadvantages

- ▶ **Slow:** thousands of training steps vs. microseconds.
- ▶ **No error guarantees** like “error $\propto \Delta x^2$ ”.
- ▶ **Local minima:** can converge to a wrong answer.
- ▶ **Hyperparameters:** architecture, learning rate, λ .
- ▶ **Smooth activations needed:** no ReLU for second-order equations.

Bottom line: simple problems in one or two dimensions: use standard methods. Networks pay off in high dimensions, complicated geometries and inverse problems. **Always compare with something.**

Read more

Parts 1–3: training a network

- ▶ **Course book** (M. Hjorth-Jensen, *Machine Learning and the Physical Sciences*, the lecture notes): Section 8.8 (backpropagation; 8.8.1 scalar chain, 8.8.4 four equations), 8.9 (vanishing gradients), 8.10 (initialisation), 8.11 (output and cost), 8.4 (universal approximation).
- ▶ **Goodfellow et al.**, *Deep Learning*: 6.5 (backpropagation), 6.2.2 (output units), 6.3, 6.4.1, 8.2.5, 8.4 (initialisation).
- ▶ **Mehta et al.**, *A high-bias, low-variance introduction to ML for physicists*, Phys. Rep. **810** (2019): Section IX.A–C.
- ▶ **Hastie et al.**, *Elements of Statistical Learning*: 11.3–11.4, 11.5.1 (starting values).

Part 4: differential equations

- ▶ **Course book**: Chapter 9: Sections 9.1–9.3 (trial solution, $\partial N/\partial x$ and why not ReLU, decay), 9.5–9.7 (Poisson, PDEs, diffusion), 9.10 (PINNs, inverse problems).
- ▶ I. E. Lagaris, A. Likas, D. I. Fotiadis, IEEE Trans. Neural Netw. **9**, 987 (1998): trial solutions.
- ▶ M. Raissi, P. Perdikaris, G. E. Karniadakis, J. Comput. Phys. **378**, 686 (2019): PINNs.

Code: every figure and number on these slides comes from `week42monday_v2.ipynb`.

Summary

- ▶ **Backpropagation** is the chain rule: compute the errors δ' from the output backwards; weight gradient = error $\times$ incoming signal.
- ▶ **Hidden activations** must be non-linear; training sees f' .
- ▶ **Output activation and cost come in pairs**, and then $\delta^L = a^L - y$.
- ▶ **Universal approximation**: a wide enough network can represent any continuous function, but the theorem says nothing about size or training.
- ▶ **Differential equations**: the residual is the cost. Conditions go into the trial solution (hard) or the cost (soft). Use smooth activations. Flexible, but slower than standard methods and without their guarantees.

Next

Tuesday: backpropagation in code. **Exercises week 42:** Friday October 16. **Project 2:** November 6.